Brain Tumour MRI Classification: TensorFlow vs PyTorch

AI Platforms, Assignment 1 | Team T10 | Alexandria University | October 2026

1. Introduction

The task in this assignment is to classify brain MRI slices into four classes (glioma, meningioma, pituitary tumour and no tumour) using transfer learning, to do it once in TensorFlow and once in PyTorch, to compare the two, and to serve one model through a FastAPI service. We trained two backbones in each framework: a ResNet50 as the main model and MobileNetV2 as a small, fast alternative. All four models use the same split, the same augmentation magnitudes and the same two-phase training recipe, so the differences between them come from the architecture, the framework and the pretrained weights, and not from different data.

Everything reported below comes from the two notebooks we ran (`tensorflow_T10.ipynb` and `pytorch_T10.ipynb`) on Kaggle with Tesla T4 GPUs. The test set was only used for reporting. Early stopping, learning-rate scheduling and the choice of which model to export all used the validation set.

2. Dataset and split

We used the Kaggle Brain Tumor MRI dataset, which ships with a *Training* folder and a *Testing* folder. We kept the official *Testing* folder untouched as the test set and split *Training* 85/15 into train and validation, stratified by class, with seed 42. The split was written to `splits.csv` by the TensorFlow notebook and read by the PyTorch notebook, so both frameworks see exactly the same images in each subset. The classes are perfectly balanced in every subset.

Subset	Glioma	Meningioma	No tumour	Pituitary	Total
Train	1190	1190	1190	1190	4760
Validation	210	210	210	210	840
Test	400	400	400	400	1600
All	1800	1800	1800	1800	**7200**

Table 1. Number of images per subset and class.

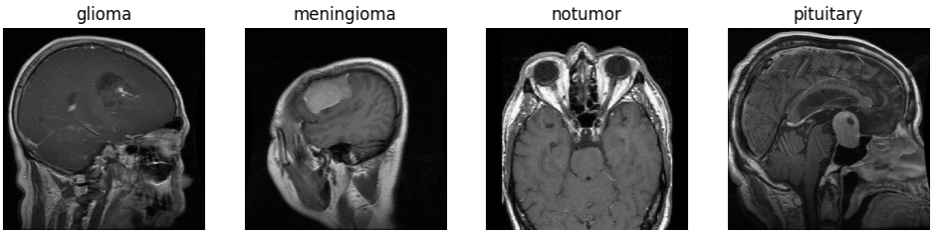


Figure 1. One training example per class (glioma, meningioma, no tumour, pituitary).

3. Preprocessing and augmentation

Every image is converted to three channels (some scans are stored as grayscale), resized to 224 x 224 with bilinear interpolation and kept as raw 0 to 255 floats. The input scaling is done inside the model, as its first layer, so the service only has to resize the upload and nothing else. TensorFlow rescales to

[-1, 1] (the ResNet50V2 and MobileNetV2 convention) and PyTorch applies the ImageNet mean and standard deviation, which is what the torchvision weights expect.

Augmentation is applied to the training images only and uses small geometric and contrast changes. We did not use horizontal or vertical flips, because left and right (and the orientation of the slice) carry meaning in a brain scan. The PyTorch values were chosen to match the Keras ones.

Transform	TensorFlow (Keras layers)	PyTorch (torchvision v2)
Rotation	RandomRotation(0.05), about +/-18 deg	RandomAffine(degrees=18)
Translation	RandomTranslation(0.05, 0.05)	translate=(0.05, 0.05)
Zoom	RandomZoom(0.1)	scale=(0.9, 1.1)
Contrast	RandomContrast(0.1)	ColorJitter(contrast=0.1)
Border fill after a transform	reflect (Keras default)	black (zero)
Input scaling (inside the model)	x / 127.5 - 1	(x - mean) / std, ImageNet
Where augmentation runs	layers inside the model, on the GPU	in the DataLoader workers, on the CPU

Table 2. Preprocessing and augmentation in the two notebooks.

4. Models and transfer learning setup

Architecture

Both frameworks use the same head design on top of an ImageNet-pretrained backbone with its classifier removed: global average pooling, dropout (0.3), then one dense layer with four outputs. In TensorFlow the dense layer has a softmax; in PyTorch it outputs logits and the softmax is applied at prediction time. The main backbone is ResNet50V2 in Keras and ResNet50 (torchvision v1.5 weights) in PyTorch, which is the closest match available, but not the same network. The second backbone is MobileNetV2 in both. Each ResNet model has about 23.6 million parameters and each MobileNetV2 model about 2.3 million, roughly ten times fewer.

Two-phase training

In phase 1 the whole backbone is frozen and only the new head is trained for 5 epochs at a learning rate of 1e-3. In phase 2 the top 40 backbone layers are unfrozen and the model is trained for up to 15 more epochs at 1e-5. BatchNorm layers stay frozen in both phases, which keeps the pretrained statistics fixed, the usual precaution when fine-tuning with a batch of 32. Both phases use Adam and cross-entropy loss, *ReduceLROnPlateau* (factor 0.3, patience 2) and early stopping on validation loss (patience 4, best weights restored). In practice early stopping never fired: every model ran all 20 epochs, and the weights that were kept are the ones from the epoch with the lowest validation loss.

Setting	TensorFlow	PyTorch
Backbones	ResNet50V2, MobileNetV2	ResNet50 (v1.5), MobileNetV2
Head	GAP, Dropout 0.3, Dense(4, softmax)	GAP, Dropout 0.3, Linear(4) (logits)
Parameters (ResNet / MobileNetV2)	23.57 M / 2.26 M	23.57 M / 2.26 M
Optimiser, loss	Adam, sparse categorical cross-entropy	Adam, cross-entropy
Batch size, seed	32, 42	32, 42
Phase 1 (head only)	5 epochs, lr 1e-3	5 epochs, lr 1e-3
Phase 2 (fine-tune)	up to 15 epochs, lr 1e-5	up to 15 epochs, lr 1e-5
Layers unfrozen	last 40 Keras layers	last 40 leaf modules
LR schedule	ReduceLROnPlateau, x0.3, patience 2	ReduceLROnPlateau, x0.3, patience 2
Early stopping	val loss, patience 4, restore best	val loss, patience 4, restore best
Software	TensorFlow 2.20.0, Keras 3.13.2	PyTorch 2.11.0, torchvision 0.26.0
Hardware	Kaggle, Tesla T4	Kaggle, Tesla T4

Table 3. Architecture and hyperparameters.

5. Results

Loss curves

Figures 2 and 3 show loss and accuracy for each epoch, with a dashed line where fine-tuning starts. In all four runs fine-tuning is where most of the improvement happens: validation loss keeps falling once the top layers are unfrozen, steeply for the two ResNets and more slowly for MobileNetV2. The two ResNet models end with a clear gap between training and validation accuracy. In TensorFlow the training accuracy reaches 99.3% against 97.0% on validation at epoch 19, and in PyTorch 99.8% against 98.1%, so there is some overfitting, but validation loss was still at its lowest around the end of training and did not turn upward. The TensorFlow MobileNetV2 run behaves differently: during phase 1 its validation accuracy fell from 83.7% to 80.6% while training accuracy kept rising, and in phase 2 the scheduler cut the learning rate three more times (down to 2.7e-7), so it was barely learning by the last epochs and finished at about 94% training and 89 to 90% validation accuracy.

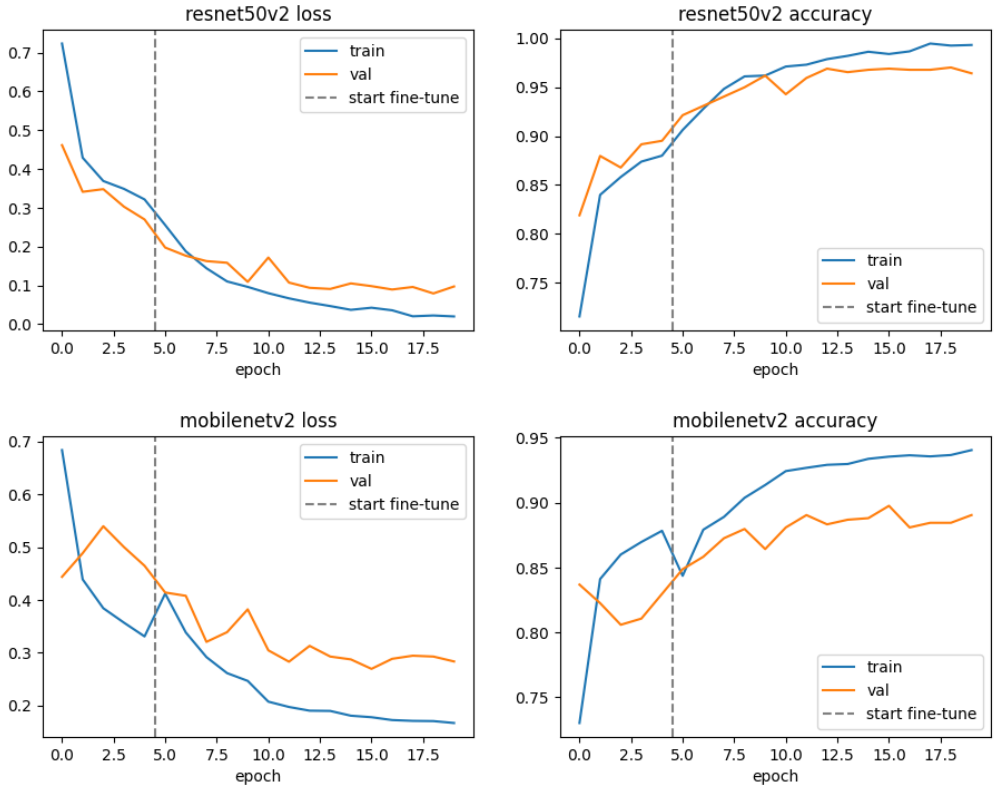


Figure 2. TensorFlow loss and accuracy per epoch. Top: ResNet50V2. Bottom: MobileNetV2.

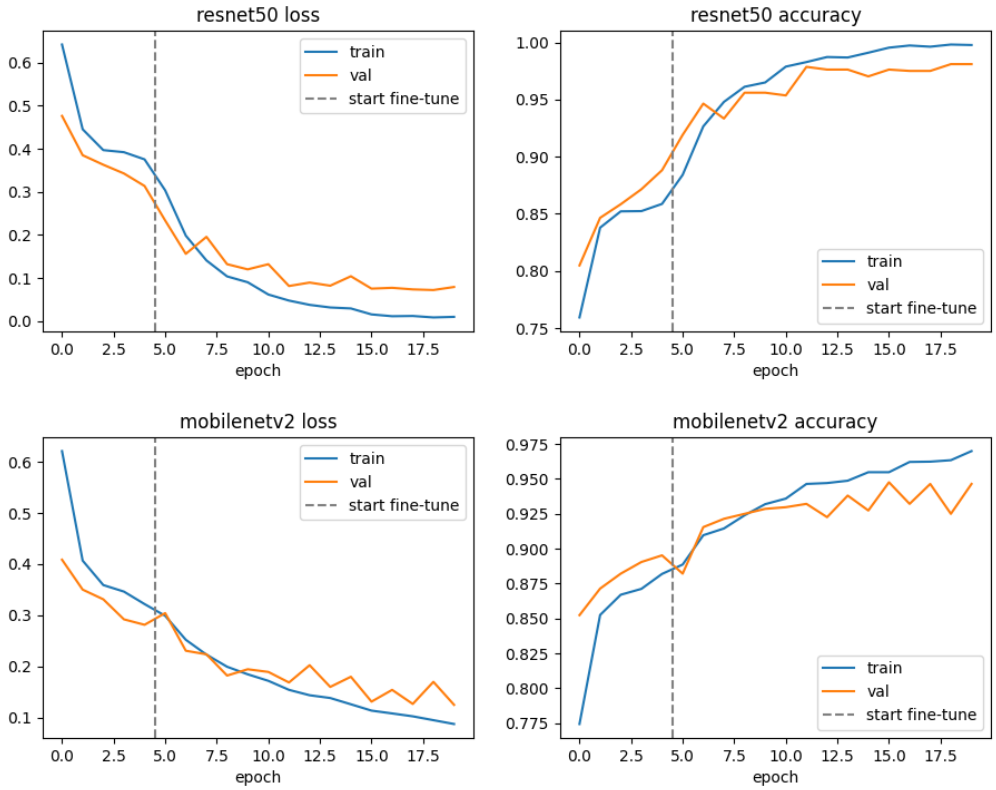


Figure 3. PyTorch loss and accuracy per epoch. Top: ResNet50. Bottom: MobileNetV2.

Test-set metrics

Precision, recall and F1 are macro-averaged over the four classes. Because the test set has 400 images per class, macro recall is equal to accuracy. The last column shows the accuracy of the same model

when only the head had been trained (end of phase 1), which is the baseline for the training-strategy comparison in Section 8.

Model	Accuracy	Precision	Recall	F1	Best val acc	Test acc, head only
TensorFlow ResNet50V2	0.9356	0.9386	0.9356	0.9343	0.9702	0.8512
TensorFlow MobileNetV2	0.8419	0.8520	0.8419	0.8376	0.8976	0.7769
PyTorch ResNet50	**0.9444**	**0.9471**	**0.9444**	**0.9433**	**0.9810**	0.8131
PyTorch MobileNetV2	0.9100	0.9115	0.9100	0.9080	0.9476	0.8494

Table 4. Test-set metrics (1600 images). Bold marks the best value in each column.

Per-class results show where the models struggle. No-tumour and pituitary cases are almost always right. Glioma has the lowest recall in every model, and its errors are mostly spread over meningioma and, more worryingly, no tumour.

Class	TF R50V2 P	R	F1	PT R50 P	R	F1
Glioma	0.9787	0.8050	0.8834	0.9851	0.8250	0.8980
Meningioma	0.8736	0.9500	0.9102	0.8912	0.9625	0.9255
No tumour	0.9412	1.0000	0.9697	0.9390	1.0000	0.9685
Pituitary	0.9611	0.9875	0.9741	0.9730	0.9900	0.9814

Table 5. Per-class precision (P), recall (R) and F1 of the two ResNet models.

Class	TF MNv2 P	R	F1	PT MNv2 P	R	F1
Glioma	0.9517	0.6900	0.8000	0.9451	0.7750	0.8516
Meningioma	0.7787	0.7125	0.7441	0.8430	0.8725	0.8575
No tumour	0.9068	0.9725	0.9385	0.9238	1.0000	0.9604
Pituitary	0.7709	0.9925	0.8678	0.9341	0.9925	0.9624

Table 6. Per-class precision, recall and F1 of the two MobileNetV2 models.

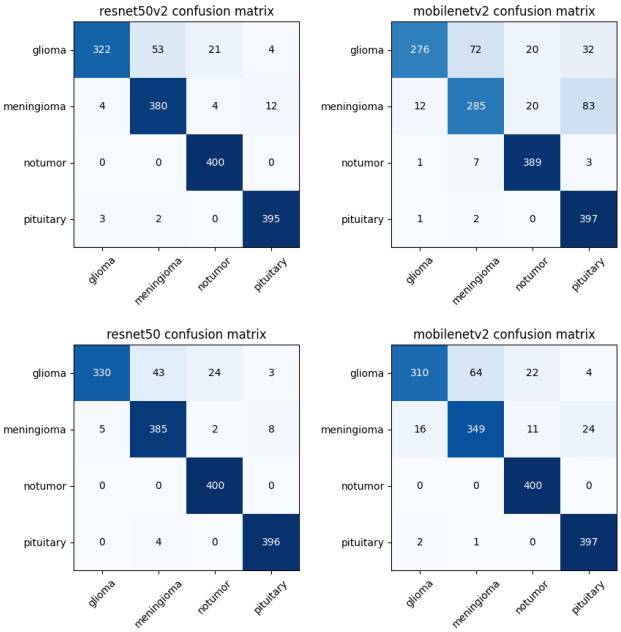


Figure 4. Confusion matrices on the test set (rows are the true class). Top row: TensorFlow ResNet50V2 and MobileNetV2. Bottom row: PyTorch ResNet50 and MobileNetV2.

In the two ResNet models roughly 5 to 6% of the glioma slices (21 in TensorFlow, 24 in PyTorch, out of 400) were called no tumour, and 53 and 43 were called meningioma. No-tumour slices were never mislabelled as a tumour. For a screening setting, a missed tumour is the more costly of the two mistakes, and we come back to this in the limitations.

6. Interpretability: Grad-CAM

We used Grad-CAM (we did not use SHAP). For a given image it takes the gradient of the predicted-class score with respect to the last convolutional feature map, averages it over the spatial positions to get one weight per channel, forms the weighted sum of the channels, keeps the positive part with a ReLU, normalises it and upsamples it to 224 x 224. The result is overlaid on the scan with the jet colour map (red means high influence). In both notebooks the final feature map is only 7 x 7, so the maps are coarse. Figures 5 to 8 show the same eight test images (two per class) for each model.

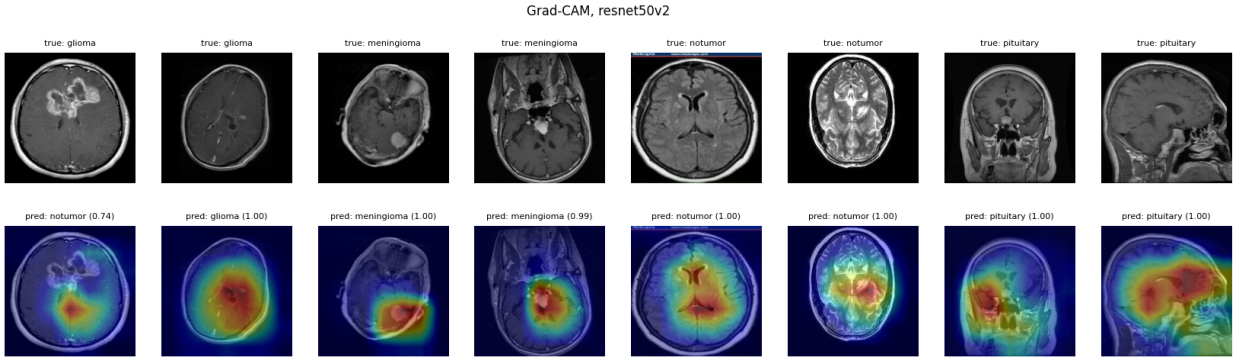


Figure 5. Grad-CAM, TensorFlow ResNet50V2. Top row: scan and true class. Bottom row: prediction, its probability and the heat map.

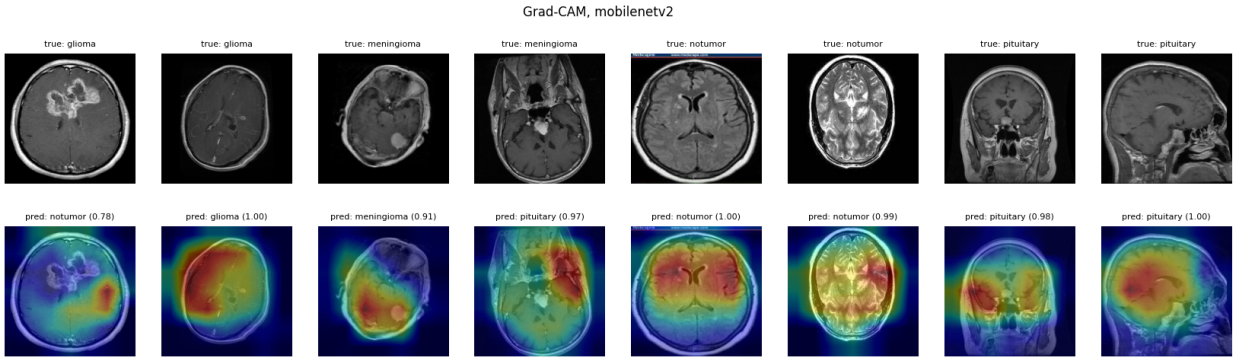


Figure 6. Grad-CAM, TensorFlow MobileNetV2.

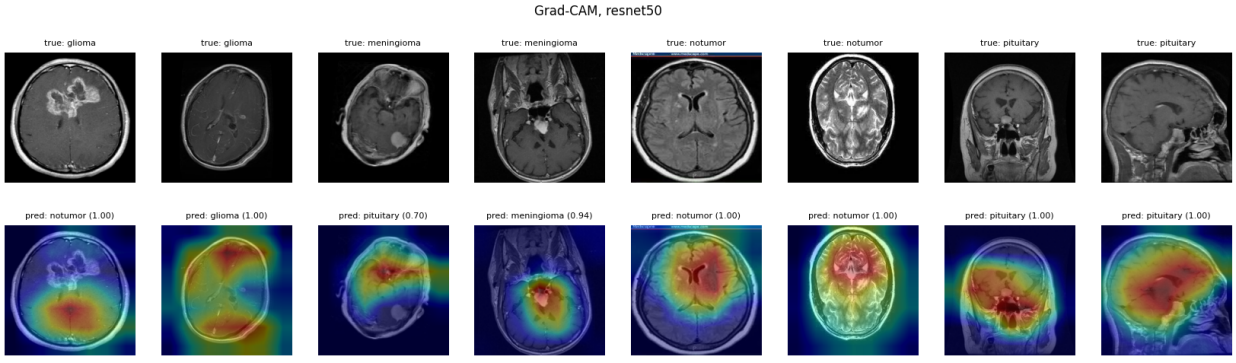


Figure 7. Grad-CAM, PyTorch ResNet50.

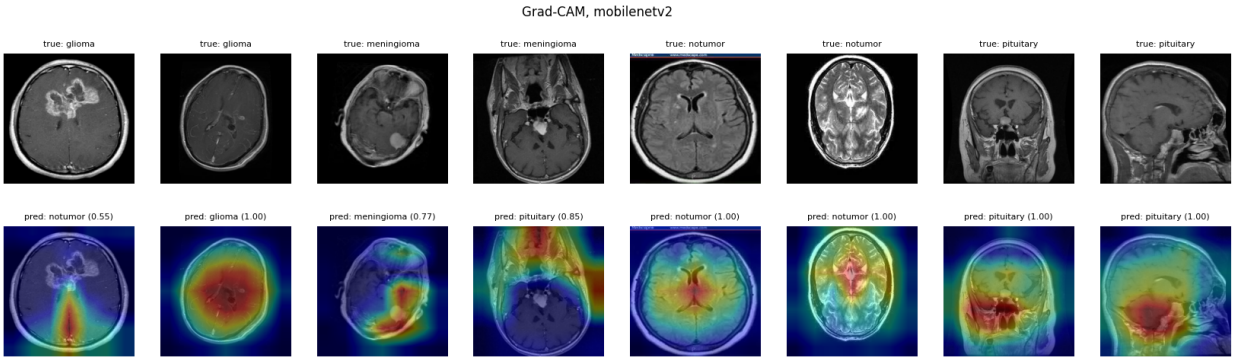


Figure 8. Grad-CAM, PyTorch MobileNetV2.

What we see. When a model is confident and right on a tumour slice, the hottest area usually sits on or right next to the lesion: the mass in the meningioma slices, and the lower central region where the pituitary sits in the pituitary slices. On no-tumour slices the heat is spread over the central brain and the ventricles, which is what we would expect when there is nothing specific to point at. This is encouraging, since the models are looking at plausible anatomy and not only at the image border or the corner text.

The failures are informative too. The first glioma slice, which has a large ring-shaped lesion in the upper part of the brain, is called no tumour by all four models, and in each case the heat falls away from the lesion, mostly in the lower or right-hand part of the slice (in PyTorch MobileNetV2 it is a thin vertical band at the bottom edge). There are three more errors among the meningioma slices: both MobileNetV2 models call the second one pituitary, with the heat on regions beside or away from the visible mass, and PyTorch ResNet50 calls the first one pituitary, but with a low probability of 0.70. On the second glioma slice the heat is broad and in several models covers a large part of the head, even though the lesion is small, so the models may be using global appearance in some cases. These are eight images per model, so they only illustrate behaviour. The model ranking is based on the 1600 test images, not on this figure (for example, TensorFlow ResNet50V2 gets 7 of these 8 right and PyTorch ResNet50 gets 6, the opposite of the full test result).

7. Framework comparison

The table compares the two frameworks as we used them. Backbone weights differ, so the accuracy difference is not purely a framework effect.

Aspect	TensorFlow / Keras	PyTorch
Training code	model.fit with callbacks. Short and quick to write.	Explicit loop, scheduler and early stopping written by hand. More code, but every tensor is easy to inspect.
ResNet backbone	ResNet50V2 (pre-activation)	ResNet50 v1.5, different pretrained weights
Input scaling	[-1, 1]	ImageNet mean / std
Augmentation	Layers inside the model (GPU), reflect fill	CPU workers, black fill
Mean time per epoch, ResNet / MobileNetV2	28.1 s / 11.9 s	26.4 s / 18.3 s
GPU latency (T4, batch 1), ResNet / MobileNetV2	8.9 ms / 5.1 ms	7.3 ms / 4.9 ms
Test accuracy, ResNet / MobileNetV2	93.56% / 84.19%	94.44% / 91.00%
Export format	.keras file, 216.8 MB (ResNet)	TorchScript .pt file, 94.7 MB (ResNet)
Loading the exported model	Needs Keras and TensorFlow	Needs only torch, no class definition
Serving parity check (PIL path vs training pipeline)	93.94% vs 93.56%	94.44% vs 94.44%

Table 7. Framework comparison. Times are from the Kaggle runs on Tesla T4 GPUs.

Accuracy. PyTorch ResNet50 is 0.9 points ahead of TensorFlow ResNet50V2 on the test set and 1.1 points ahead on the best validation accuracy. With 1600 test images the standard error of an accuracy near 94% is about 0.6 points, so this gap is small compared with the noise, and we only trained one seed per model. We would not claim that PyTorch is better at this task from these runs. The MobileNetV2 gap (6.8 points) is much larger. A likely reason is the learning-rate behaviour described in Section 5: the Keras scheduler lowered the rate repeatedly on a noisy validation loss, and the model effectively stopped learning, while the PyTorch run kept the rate at 1e-5 throughout. We did not test this directly.

Speed. The epoch times are not a clean framework comparison. PyTorch MobileNetV2 takes about 18 s per epoch in both phases, frozen or not, which suggests it is limited by the CPU data pipeline (decode, resize and augment in two workers) and not by the GPU. In TensorFlow the augmentation runs on the GPU inside the model. For the ResNet, where the GPU work dominates, the two are close. Inference latency on the GPU favours PyTorch slightly for both backbones.

Serving. This was the clearest practical difference. The TorchScript file is less than half the size of the Keras file, can be loaded with just `torch.jit.load`, and gave exactly the same accuracy through the serving path (PIL resize) as through the training pipeline. The Keras model changed by 0.4 points, which comes from the different bilinear resize implementations in `tf.image.resize` and PIL.

8. Architecture, hyperparameter and training strategy comparison

ResNet50 vs MobileNetV2

MobileNetV2 has about a tenth of the parameters (2.26 M against 23.57 M) and is faster, but it was clearly less accurate in both frameworks. On the test set it lost 9.4 points in TensorFlow and 3.4 points in PyTorch relative to the ResNet. Its GPU latency was lower by 2.4 to 3.8 ms, which is small in absolute terms for a single image. Most of its loss comes from meningioma and glioma recall (Table 6), the two classes that look most alike. For this task, where a mistake has a real cost, the extra

accuracy of the ResNet is worth the extra size.

Model	Params (M)	s / epoch	GPU latency (ms)	Test acc	Test F1
TF ResNet50V2	23.57	28.1	8.9	0.9356	0.9343
TF MobileNetV2	2.26	11.9	5.1	0.8419	0.8376
PT ResNet50	23.57	26.4	7.3	0.9444	0.9433
PT MobileNetV2	2.26	18.3	4.9	0.9100	0.9080

Table 8. Cost and accuracy of the two architectures.

Training strategy: head only vs fine-tuning

The test accuracy after phase 1 (frozen backbone) can be compared with the final accuracy after phase 2. Fine-tuning added 8.4 points for TensorFlow ResNet50V2, 6.5 for TensorFlow MobileNetV2, 13.1 for PyTorch ResNet50 and 6.1 for PyTorch MobileNetV2. Features learned on natural ImageNet images are a reasonable start for MRI, but not good enough alone: the frozen backbone gives 78 to 85% and fine-tuning the top layers gets the ResNets above 93%. One result worth noting is that after phase 1 the PyTorch MobileNetV2 (84.9%) was ahead of the PyTorch ResNet50 (81.3%), so the larger network only wins once its upper layers are adapted.

Model	Head only (phase 1)	After fine-tuning	Gain (points)
TF ResNet50V2	0.8512	0.9356	+8.4
TF MobileNetV2	0.7769	0.8419	+6.5
PT ResNet50	0.8131	0.9444	+13.1
PT MobileNetV2	0.8494	0.9100	+6.1

Table 9. Test accuracy before and after fine-tuning.

Hyperparameters

We used one set of hyperparameters for every model (Table 3) and did not run a search, so we cannot say these values are best. The choices were the usual defaults for this recipe: a large learning rate for the new head, a rate 100 times smaller when the backbone is unfrozen, frozen BatchNorm and moderate dropout. The runs suggest two things worth trying next. A smaller reduction factor or a longer patience for the scheduler (the TensorFlow MobileNetV2 case), and a different number of unfrozen layers than 40. Note also that '40 layers' is not the same depth in both frameworks, since Keras counts layers and our PyTorch code counts leaf modules.

9. Limitations

Possible data leakage. The dataset is made of 2D slices without patient identifiers. Slices from the same patient can end up in both the training and test folders, and the validation split is random on the training folder, so near-duplicate slices may sit in train and validation as well. This would make all our scores optimistic.

Data source. The data is one public compilation from several sources, with uncontrolled scanners and protocols, and we could not verify the labels clinically. There is no external test set from another hospital, so we do not know how the models generalise.

Scope. Only four classes and only 2D slices. A real system would deal with other findings, other sequences and full 3D volumes.

Statistical strength. Each model was trained once with one seed. As computed in Section 7, the difference between the two ResNet models is within the noise, and we have no measurement of run-to-run variance.

Error type. About 5 to 6% of glioma test slices were classified as no tumour. A tool that misses tumours at this rate is not safe to use for screening, and accuracy alone hides this. We did not tune the decision threshold or use class-specific costs.

Interpretability. The Grad-CAM maps are 7 x 7 upsampled, so localisation is coarse, and they are qualitative evidence only. A map that looks plausible does not prove the model reasons correctly.

Comparison fairness. The two frameworks do not use identical backbones, weights or augmentation fill, and the layer counts for fine-tuning differ. Training times come from shared Kaggle machines. The latency numbers were measured on a GPU, whereas the deployed service runs on the CPU, so the latency each request reports (Section 11) is the figure that matters for the service.

Intended use. This is a coursework model and is not a diagnostic tool.

10. Deployed model and why we chose it

We deployed the **PyTorch ResNet50**, exported as a TorchScript file (`model_T10.pt`), behind a FastAPI service. The selection was made on validation results, as the notebooks do, and the other criteria agree with it.

Criterion	PyTorch ResNet50	TensorFlow ResNet50V2	PyTorch MobileNetV2
Best validation accuracy	**0.9810**	0.9702	0.9476
Test accuracy / F1	**0.9444 / 0.9433**	0.9356 / 0.9343	0.9100 / 0.9080
GPU latency (batch 1)	7.3 ms	8.9 ms	4.9 ms
Model file	94.7 MB	216.8 MB	not exported
Serving parity	exact	0.4 points off	not checked
Dependencies to serve	torch only	TensorFlow + Keras	torch only

Table 10. Candidates for deployment.

Accuracy was the main criterion, because the cost of a wrong prediction is much higher than the cost of a few extra milliseconds. PyTorch ResNet50 has the best validation and test scores, a file less than half the size of the Keras model, no dependency on a model class when loading, and no difference between the serving and training pipelines. MobileNetV2 is faster (4.9 ms against 7.3 ms), but it is 3.4 points less accurate, and for a single-image request that difference in latency is not noticeable.

The service (`app/main.py`) runs on the CPU and exposes: `GET /health` (status, framework and class names), `POST /predict` (label, confidence, per-class probabilities and server-side latency), and `POST /explain` (a Grad-CAM overlay as a base64 PNG for the predicted class). It also serves a small web page at `/` and an offline Swagger page at `/docs`. Uploads are checked from the file bytes, not the file name: only JPEG and PNG are accepted, files larger than 10 MB are rejected, and empty or corrupted files return a clear error instead of a server crash. The /explain endpoint scores the class before the softmax, which differs slightly from the notebooks, where the softmax probability was used.

11. Example API requests

Both examples use curl on a local server (`uvicorn app.main:app --port 8000`). The JSON below is exactly what the service returns, and neither response depends on the model weights.

Example 1: health check

```
curl http://localhost:8000/health
```

Response (HTTP 200):

```
{
  "status": "ok",
  "framework": "pytorch",
  "classes": [
    "glioma",
    "meningioma",
    "notumor",
    "pituitary"
  ]
}
```

Example 2: a request the service rejects (a text file sent as an image)

```
curl -X POST http://localhost:8000/predict -F "file=@notes.txt"
```

Response (HTTP 415):

```
{
  "detail": "'notes.txt' is not an image. Send a JPEG or PNG file."
}
```

Sending an empty file returns HTTP 400 with the message *The uploaded file is empty. Send a JPEG or PNG image in the 'file' field.*, and a file over 10 MB returns HTTP 413.

A successful classification uses the same endpoint with a scan, for example `curl -X POST http://localhost:8000/predict -F "file=@scan.jpg"`. It returns `framework`, `label`, `confidence` (0 to 1), `probabilities` (one value per class, summing to 1) and `latency_ms` (preprocessing plus inference time on the server).